The System Terminal

44helper includes a root shell that runs in the browser. This summary covers how it is switched on, who may open it, how a session is opened and ended, what runs inside it, and the protections that apply.

OFF BY DEFAULT | LAN ONLY | PASSWORD EACH TIME | ONE AT A TIME

SUMMARY · READ-ONLY REVIEW

What the terminal is

The Terminal button in the 44helper header opens a root shell on the node, in a panel of the same page. Its output is drawn by a terminal emulator that the helper serves itself. The shell is a real login bash run as root.

PART	WHAT IT IS
The switch	A file, `/etc/44helper/terminal.enable`. The terminal works only while this file exists.
The ticket	A one-time key, valid for 60 seconds, issued after the root password is checked.
The socket	A WebSocket to `/api/term/ws`, carrying the terminal's input and output.
The shell	A pseudo-terminal running `bash -l` in `/root` (or `/` if that is missing).
The emulator	xterm.js, with its fit addon and stylesheet, served from the helper at `/static/`.

The switch

2.1 The file

The terminal is off until `/etc/44helper/terminal.enable` exists. The helper accepts the file only if it is a plain file, owned by root, and not writable by anyone but root. Any other file is refused, and the page says why.

2.2 Switching on from the page

1. The page asks the helper for the terminal's status. If the terminal is off and this browser may switch it on, it asks the operator to switch it on.
2. The operator enters the root password. The helper checks it, and counts a wrong password against the same lockout as the login (five failures in five minutes lock the address for 15 minutes).
3. The helper writes the file: a short note with the address that switched it on, and the time. The file is written to a temporary name first, then renamed into place.
4. The switch is logged, with the address.

2.3 Switching on at the node

The page can only switch the terminal on if the request is allowed (Part 3), and if the helper runs as root. If either fails, the page gives the command to run on the node itself: `sudo touch /etc/44helper/terminal.enable`.

2.4 Switching off

Switching off deletes the file. An open session ends at its next check (Part 5).

The access rules

3.1 Local network only

The helper accepts a request for the terminal only from a loopback address, a link-local address, or a private address (the local network). Any other address is refused with "is not a local-network address". Addresses in 44Net are public, so a request from one is refused.

3.2 Refused through a proxy

A request that carries a proxy or tunnel header is refused, whatever its address. The headers checked are those that Cloudflare, load balancers and other proxies add (for example `Cf-Connecting-Ip`, `X-Forwarded-For` and `Via`). The page says the terminal "only works from your local network" in that case.

3.3 The page must match

The WebSocket handshake is refused unless its Origin matches the Host. A request from another site is therefore refused even from a local browser.

3.4 The root password, each time

Opening the terminal needs the root password, sent with the open request. The password is checked on every open, and the field is cleared after each attempt. Wrong passwords count against the shared lockout. A root login session alone is not enough.

3.5 Logged in

The open request also needs a logged-in session. A session that has ended ends the terminal too (Part 5).

From the Terminal button to a shell

1. **The page checks the status.** `/api/term/status` reports whether the terminal is on, whether this client is allowed, whether the terminal is busy, and which commands are on the allow-list.
2. **The operator chooses the shell, or a command.** The page offers the root shell and the commands on the allow-list (Part 6).
3. **The page sends the open request.** `POST /api/term/open` carries the command id and the root password. The helper checks the switch, the client address, the allow-list, the lockout, the password and that no other terminal is open.
4. **The helper issues a ticket.** The ticket is random, valid for 60 seconds, and bound to the session that asked for it.
5. **The page loads the emulator** from `/static/`: the stylesheet, the script and the fit addon. Each file is decompressed and checked against its SHA-256 before it is served. A file that fails its check is not served.
6. **The page opens the WebSocket** with the ticket in its address. The helper takes the ticket (it can be used only once), checks that the session matches, and claims the single terminal slot.
7. **The helper starts the command** in a new pseudo-terminal, sets its size to 80 by 24, and relays output both ways.

The ticket is used once, so a copied address cannot open a second terminal later.

While a session is open

EVENT	WHAT HAPPENS
Typing	Each key is sent as a message and written to the shell. The page sends typed text as JSON messages.
Resizing the window	The page sends the new size, and the helper resizes the pseudo-terminal (between 2 and 500 columns, 1 and 300 rows).
Closing from the page	The page asks for confirmation ("the command running in it will be stopped"), then closes the socket.
The command exits	The helper sends the exit code and closes the socket with "command finished".
Ten minutes with no typing	The helper sends an idle exit and closes the socket.
The terminal is switched off	The helper's check finds the switch file gone, and closes the socket.
The login ends	The helper's check finds the session gone, and closes the socket.

When a session ends, the helper stops the command's process group: it sends a hang-up, waits 3 seconds, then sends a kill. The helper releases the single terminal slot in all cases.

The idle timer counts from the last key typed. Output from a running program does not reset it.

The allow-list

6.1 The commands

ID	RUNS	HOW IT IS BUILT
shell	A root login shell: `cd /root 2>/dev/null || cd /; exec bash -l`	Written in the helper
asl_menu	asl-menu (ASL3 node settings)	The ASL3 install step's own command
m17_install	The USRP2M17 installer (asks for the callsign)	The M17 install step's own command
amp_firstrun	amp-server, in the foreground	The Ampersand step's own command
cr_signin	Claude Code sign-in, as the login user	The Claude Remote step's own command
cf_token	Cloudflare connector service. The token is typed hidden, and checked for the allowed characters before use.	Written in the helper
cf_quick	Cloudflare quick test tunnel, on a port the operator gives (default 8989)	Written in the helper
amp_hub_env	nano on `/usr/etc/amp-hub.env`	Written in the helper

An entry is offered only if its step is found in the install tables. A command is taken from its step's command, so the terminal runs the same command the install step would.

6.2 The shell is not limited

The root shell has no allow-list inside it. Once it is open, the operator can run any command as root. The allow-list limits which programs the terminal can start, not what happens in the shell. This is the most powerful capability on the helper, and the book should describe it as such.

6.3 Steps that need a person

The install steps marked manual in Stage 1 (Part 7) are the ones that run here: the ASL3 menu, the Ampersand first run and hub file, the Cloudflare token and quick test, and the Claude sign-in. The terminal is how those steps are done by hand.

What the terminal shows

7.1 Layers, not one lock

The terminal is protected by the switch, the address rules, the proxy rule, the origin rule, the password, the lockout, the single-use ticket and the single slot. A request must pass all of them. The book can describe the design as defence in depth, and list each layer.

7.2 The protections are for the network, not the node

The rules keep the terminal off the internet and off other sites. They do not limit a person on the local network who knows the root password. The book should say that the root password is the one secret that matters.

7.3 Embedded and checked

The emulator is inside the helper and checked by hash, so the page does not reach outside for it. This suits a node with no reliable internet.

7.4 Not checked

- The browser-side layout and the emulator's behaviour on a phone, which the page code was read for but not run.
- The exact list of addresses that Python's private-address test accepts, beyond loopback, link-local and private.
- The effect of the 44Net address rule in a network that uses another range for its LAN.